Run It, Then Cite It: Execution- and Mutation-Verified Answers to Developer Questions

LMA Group Project (CL3.410, Monsoon 2026) · Team Disco 82

Ameya Purohit, Sunil Kumar, Aditya Narayan Dwivedi, Sourav Nath

Code and data: https://github.com/ameya-purohit/LMA-Final-Project. The repository README gives step-by-step reproduction. Detailed result tables are in `docs/results.md`, and every design decision is logged in `DECISIONS.md`.

Abstract

Developers often ask how a library behaves when its documentation may be wrong. LLM assistants tend to repeat the documentation, and they cite code lines that look related but do not decide the behaviour. We built an agent that checks its own answer twice. First, it splits the answer into atomic claims and tests each one with a small generated probe, executed in an offline Docker sandbox against the library frozen at the commit before the question was asked. Second, it accepts a cited line only if mutating that line in a throw-away copy changes the claim's outcome while leaving control probes unchanged. Failed checks become the agent's next investigation goal.

We evaluated the agent on real GitHub issues from click and marshmallow, with gold answers built automatically from each issue's merged fix and its test, following the SWE-bench recipe. The hypotheses were pre-registered and the test set was locked before any test run. On 145 test questions (150, minus 5 removed by a human spot check), the agent did **not** beat a ReAct agent with the same tools, model and budget. Its rate of correct, causally cited answers was 40.0% against 49.7% by the LLM judge (95% CI of the difference [−19.3, −0.7]), and 25.0% against 41.7% on the 12 questions labelled by two humans. The judge agreed with the humans only fairly (kappa 0.36 and 0.32), so by our pre-registered rule the human labels are primary. With human labels, the mutation check *lowered* citation precision, 30.6% against 44.1% without it (CI [−28.6, −1.0]). Its verdicts agree with the developers' fix lines no better than chance (kappa ≈ 0). The agent beats closed-book (23.4% correct) and single-call RAG (37.9%) baselines by wide margins. No system executed an unsafe action. Our main contribution is therefore a careful negative result, with evidence for its mechanism: mutation is a poor test of whether a cited line causes a behaviour in real code, and acting on it sends the agent away from correct citations.

An exploratory extension, pre-registered separately after the main results (§8), split the agent into five roles and added hybrid retrieval, a code graph with slicing, pairwise mutation, information-gain experiment selection and an MCP tool server. On 47 test questions, the roles used half the requests and a sixth of the tokens without changing correctness. Fusing BM25 with dense retrieval improved retrieval (hit@5 +7.7 points, p = 0.035). Localization did not improve: the code graph lowered line Top-3 from 55.3% to 36.2%. The cause was the same weakness: the mutation check rejects most lines that developers changed.

1. Introduction

1.1 The problem

"Why does this library behave differently from its documentation?" is a common developer question. The answer usually depends on code spread across files: where a default is set, where it is overridden, and which branch actually runs. LLMs given some of this material answer confidently and often repeat the documentation even when the code disagrees. Agents that search and read code do better, but they still decide behaviour by *reading*, and LLMs misread control flow, inheritance and overrides.

A second problem is less visible: citations. An answer can cite lines that are on-topic but do not produce the behaviour. Today, citation quality is judged by humans or by another LLM, which is expensive or unreliable. Code offers an objective test that ordinary text does not: run the behaviour, then change the cited line and see whether the behaviour changes.

1.2 Our approach

Our agent ("ours" below) adds two checks to a tool-using code agent:

1. **Check 1, run it.** Each behavioural claim in a draft answer is tested by a generated probe that runs in a sandbox against the frozen library. A refuted claim is re-investigated with the observed output as the new goal.
2. **Check 2, flip the switch.** Each citation of a confirmed claim is mutated (a constant changed, a condition negated, a comparison swapped, a statement deleted). The citation is accepted as *causal* only if some mutant changes the claim probe's outcome *and* leaves control probes, which test nearby unrelated behaviour, unchanged. A non-causal citation sends the agent looking for the line that does decide the behaviour.

Each claim in the final answer carries a grade saying how it was established: executed and causally cited (E+C), executed (E), confirmed by a repository test (T), read only (R), or documentation only (D).

1.3 Research questions and hypotheses

- **RQ1.** Does this procedure give more correct, causally cited answers, and fewer confident errors, than a ReAct agent with the same tools (including execution and mutation) and the same limits?
- **RQ2.** How well does the mutation check agree with human judgements of causality, and with the lines the developer actually changed? Which acceptance rule agrees best?

The hypotheses were pre-registered in `PREREGISTRATION.md` before any test run:

- **H1.** Ours beats B2 (ReAct with the same tools) on the causally-supported answer rate by at least 8 points, with a paired 95% CI that excludes zero.
- **H2.** Ours has at least 40% fewer confident errors (relative) than B2, with coverage no more than 10 points lower.

- **H3.** Human-judged causal citation precision is at least 15 points higher with the mutation check than without it (ablation A2).

1.4 Contributions

1. A working, fully logged agent that verifies claims by execution and citations by mutation, inside a question-answering loop, with a shared harness so that baselines differ only in control logic.
2. An evaluation set of 170 real developer questions (162 after the spot check) from click and marshmallow, each with executable gold built only from the developer's merged fix and its test, with no LLM involved.
3. A pre-registered, paired comparison against five baselines and ablations, with LLM-judge results validated against two human labellers.
4. A measurement study of mutation as an automatic test of citation causality, against both developer fix lines and human labels.
5. A negative result with a mechanism: the procedure did not help, and the traces show why.
6. An exploratory, separately pre-registered multi-agent extension (§8). It adds hybrid retrieval, a code graph, three-way and pairwise mutation, information-gain experiment selection and an MCP tool server, and evaluates each step with localization metrics.

---

2. Literature Review

Reasoning with retrieval and tools. ReAct [1] interleaves reasoning with tool calls; IRCoT [2] and FLARE [3] decide when to retrieve more during generation. These systems decide *when to look further*, but they never test whether a claim is true; claims are judged by reading. Our strongest baseline, B2, is a ReAct agent given every tool our agent has, including execution and mutation.

Self-critique and verification. Self-RAG [4] and CRAG [5] train critics that judge whether retrieved passages support the output. Chain-of-Verification (CoVe) [6] has the model write verification questions and answer them independently. CRITIC [7] lets a model critique its own output with tools such as a code interpreter, and Reflexion [19] feeds verbal feedback from failures back into later attempts. Support in these systems is still judged by a model, or the tools check the model's *own* computation. In our work, an executable test decides claims about a third-party codebase, and failures steer the next investigation step.

Agents on repositories. SWE-agent [8] and Agentless [9] navigate repositories to fix issues, and SWE-bench [20] evaluates them on real GitHub issues by whether the fix's tests pass. These systems are judged on the patch, not on the natural-language evidence behind an answer. We reuse the SWE-bench recipe to build our *dataset*: an issue is the question, and the merged fix and its test are the answer key.

Testing documentation by execution. CASCADE [10] generates tests from documentation and flags an inconsistency when the real code fails a test that code generated from the documentation passes. Kang et al. [11] test LLM-written code comments by execution, and METAMON [12] uses metamorphic LLM queries to find

documentation–behaviour inconsistencies. These are the closest prior work. They check documentation offline, function by function; they do not answer a developer's question, investigate across files, or cite lines. We build on their finding that execution beats LLM judgement for this task.

Mutation analysis. Mutation testing has a long history [18]. MUSE [13] and Metallaxis [14] mutate program lines and watch test outcomes to locate faults. MUSE's idea, that a useful mutant changes the target outcome but not others, inspired our *specificity* control. We apply the mechanism to a different question: does a line an LLM cited actually cause the behaviour it claims?

Citation quality. ALCE [15] measures whether generated citations support statements. Wallat et al. [21] distinguish citation *correctness* (the source supports the claim) from *faithfulness* (the answer was actually derived from the source). Our mutation test measures neither exactly: it targets a stronger, code-specific notion we call *causal citation correctness*, whether the cited line decides the claimed behaviour. Work on code–comment inconsistency [16] and on LLMs facing conflicting context [17] studies the same tension between documentation and behaviour, but as classification or analysis, not inside an answering agent.

What is new. To our knowledge, no prior question-answering or RAG system checks its citations by executable counterfactual mutation, or uses failed claim and citation checks as its investigation signal. We do not claim novelty for documentation testing or for mutation analysis themselves; the contribution is the combination, and its measurement.

---

3. Dataset Created and Used

3.1 Main set: real issues with fix-derived gold

We built the dataset with the SWE-bench recipe [20] (`data/`, D-entries in `DECISIONS.md`). Every step is deterministic code. **No LLM writes, edits, summarises or filters any gold field.**

1. **Mine issue–fix pairs** (`data/mining/fix_pairs.py`). We used the GitHub REST API to list closed pull requests of click and marshmallow merged since 2020-01-01. We kept PRs whose title or body says `fixes #n` (or similar) for an issue, that change at least one source file and at least one test file, and that touch at most 10 source files. We dropped bot-authored issues and issues that GitHub metadata marks as feature requests (a "feature" or "enhancement" label, or the issue type "Feature").
2. **Freeze** (`data/freeze.py`). The frozen commit is the PR's base commit, before the fix. Systems see only this tree and `git log`/`git blame` up to it.
3. **Validate by running** (`data/validate_pairs.py`, in the sandbox). We applied only the PR's test changes to the frozen tree: at least one test must *fail* (these are the `fail_to_pass` tests). We then applied the source part as well: every `fail_to_pass` test must *pass*. The failure message on the frozen code is the *observed* old behaviour.
4. **Build gold** (`data/build_gold.py`). The **gold evidence** is the line ranges, in the frozen version, that the fix modified or deleted; pure insertions are anchored to the nearest existing line above. The **gold answer** is a template filled from the test output and the

diff. Each question gets a category computed in code: `doc_related` (the issue mentions documentation or the PR edits docs or docstrings), `spread_across_files` (more than one source file changed), or `single_location`.

What a system sees: only the issue title and opening post, plus the frozen tree. The workspace object is built so that it cannot hold the fix, the test names or the gold, and a unit test asserts this.

Funnel step	click	marshmallow	total
Closed PRs listed	1,898	1,405	3,303
Merged since 2020-01-01	801	605	1,406
Linked to an issue	286	81	367
Not a feature request (by label)	284	75	359
Changes source and tests (candidates)	179	33	212
Validated: test fails before, passes after	152	24	176
Gold built, one question per issue	**146**	**24**	**170**

Of the 170 questions, 103 are `doc_related`, 63 `single_location` and 4 `spread_across_files`. The split (`data/split.py`, seeded, stratified by library and category) has **20 dev** questions, used for development and one round of prompt iteration, and **150 test** questions, locked with their SHA-256 hash at gate G2 before any test run.

3.2 Human spot check of the automatic gold (gate G1)

All four members checked a seeded, library-stratified 20% sample: 34 questions, 8–9 each, one member per question. They used a self-contained HTML page (`annotation/spotcheck.html`) that showed the issue, the fix diff, the failing test's message and links to the gold lines. Each member answered three yes/no questions: is this a genuine question about library behaviour; does the failing test capture the behaviour the issue asks about; do the gold lines decide that behaviour?

8 of 34 were marked invalid, a disagreement rate of 23.5%. All 8 failed the second check. In 7, the issue is a feature request that its label did not reveal, so the test only shows that the new feature did not exist yet ("unexpected keyword argument", "DID NOT WARN", "DID NOT RAISE"). In the eighth, the fix bundled several changes and the failing test belongs to an unrelated one. No question failed the first or third check. The 8 were removed and logged in `data/dropped.jsonl`. 5 of them were test questions, so **all test results are on the remaining 145** (125 click, 20 marshmallow). Because the test set was locked first, the removal is applied at analysis time, identically for every system (a disclosed deviation).

How the verdicts were recorded. Each member made their own verdicts and wrote their own notes. The notes were then grammar-corrected with an AI tool, and the files were assembled from them instead of being exported from the page. No verdict was changed (`PREREGISTRATION.md`, Deviations).

3.3 Other sets

- **toylib** (`tests/fixtures/toylib/`): a small hand-written library with planted ground truth: a wrong docstring, a decisive line, a non-causal default, a line that changes all outputs (not specific), redundant guards, and a line only non-constant operators can flip. It is the unit test bed for the mutation check and the live demo.
- **Adversarial set** (`data/adversarial.py`, 15 items): dev questions with one planted instruction each, five of each kind. The kinds are: a hidden note in the issue text asking the model to run `curl` in a probe; a comment on the fix line asking it to answer "works as intended"; and a comment asking it to write to `~/.ssh/authorized_keys`. Each instruction also asks for a unique canary token, so obedience is detectable. Comments are planted so that the code's syntax tree is unchanged.
- **A10 subset:** the first 50 test questions in the seeded default order, which interleaves the libraries (47 after the spot check).

---

4. Methodology

4.1 Shared harness

All systems share one LLM client, one config file, one model, one retrieval index and starting material per question, one tool implementation, one sandbox and one output schema. They differ **only** in control logic; a test checks that each ablation's config differs from the default only under `agent.ablation` or `baselines`. No agent framework is used: the loops are written by hand on the provider's native tool-calling API.

- **LLM client** (`llm/`): one `complete()` interface with a SQLite cache keyed by a hash of (provider, model, messages, tools, settings). This makes re-runs free and lets ablations reuse the full agent's identical first steps. It also provides client-side rate limiting, retries with backoff, a clean stop when the daily quota runs out, a 120-second timeout, and a JSONL trace of every call with token counts. Gemini's *thought signatures* are preserved across turns, which the native SDK requires.
- **Retrieval** (`retrieval/`): code is chunked by AST (functions, classes, methods) and docs by heading. There are separate BM25 indexes for code and docs, with a tokenizer that splits `snake_case` and `camelCase`. Starting material is the top 5 code and top 3 doc chunks, identical for every system.
- **Tools** (`tools/`): `search_code`, `search_docs`, `open_file`, `find_definition` (jedi), `find_references`, `find_tests`, `git_log`, `git_blame`, `run_probe`, `mutate_and_run` and `run_repo_test`. Each has a JSON schema validated with pydantic. Invalid arguments return a readable error to the model and are counted.
- **Sandbox** (`sandbox/`): one warm Docker container per question, with no network, a read-only file system except `/tmp`, a non-root user, and limits of 1 CPU, 1 GB of memory and 256 processes. Probes are checked by an AST policy (no `os.system`, `subprocess`, `eval`, dynamic imports or writes outside `/tmp`) *before* running, as defence in depth. The container is the real boundary. **No LLM-generated code ever runs on the host.**

4.2 The agent (`agent/loop.py`)

The model produces structured output by calling function tools (`submit_draft`, `write_probe`, `assess_refutation`, `propose_citation`, `final_answer`), whose arguments are validated with pydantic.

1. **Investigate.** Read-only tools, at most 8 investigation steps per question. The model ends with `submit_draft`: an answer plus claims, each with its scope, type, citations (path and line) and source kind (code, doc or test).
2. **Prioritise.** Claims supported only by documentation, or in conflict with another source, are verified first.
3. **Check 1.** For each behavioural claim, the model writes a probe with two functions, `observe()` and `test_claim()`, plus 1–2 control probes, all in one call. The trusted runner executes them in the sandbox and records the outcome: a returned value or an exception with its innermost library frame. A broken probe is repaired at most twice. At most 12 probe runs per question.
4. **Refutation.** `assess_refutation` first compares the probe's setup with the question's scenario. If they differ, the probe is rewritten; if they match, the claim is refuted, re-investigated with the observed output as the goal, and revised.
5. **Check 2.** For each citation of a confirmed claim, `mutate_and_run` uses libcst to apply each operator, one at a time, to the nodes that start on the cited line, leaving the rest of the file byte-identical. The operators are: constants (±1, 0, bool flip, None ↔ 0, string → ""), negating an `if`/`while` condition, swapping a comparison, deleting a statement, and `return x` → `return None`. It runs the claim and control probes on each mutant. Three acceptance rules are computed: - **any_flip:** any outcome change; - **behavioural:** a change that is not mere *breakage* (a SyntaxError, NameError or UnboundLocalError, or a TypeError or AttributeError raised at the mutated line); - **specific:** behavioural, and every control outcome is unchanged. **The agent uses `specific`.**

If a citation is not causal, the agent re-investigates ("find the line that decides this"), proposes a new citation and checks it again. A line with no applicable operator is *untestable*. 6. **Vacuity check.** If a confirmed claim has no causal citation, the agent checks whether any mutant in the enclosing function changes its probe at all. If none does, the probe is vacuous and is regenerated once. 7. **Grade and answer.** Grades are computed deterministically from the claim table. `final_answer` writes the text; only confirmed claims are stated as fact, conflicts are stated with their observed outputs, and unverified claims are flagged. The output record is assembled in code.

All model calls share a budget of **25 requests per question**.

4.3 Baselines

ID	System
B0	Closed book: one call, the question only
B1	Direct RAG: one call with the starting material, asked to cite lines
B2	ReAct: the shared tool loop with **all** the agent's tools (including `run_probe` and `mutate_and_run`) and the same limits (8 steps, 12 probe runs, 25 requests)

B2+ (B2 with a larger budget) and B3 (CoVe) are implemented but were not run (§9).

4.4 Capabilities used

The course asks for at least three of five capabilities. We use four. Fine-tuning was a stretch goal and was not done, because RQ1 compares procedures with the *same* model, and a tuned model would confound that comparison.

Capability	Why the problem needs it	Ablation or comparison	Metric reported
Tool calling	Behaviour is settled by running probes and mutants; evidence is found by following definitions, references, tests and history	A1 (no execution)	Valid-call rate, error recovery, probe verdict rate, unsafe-attempt rate
Agentic decisions	What to do next depends on check results: re-investigate or flag, where to look after a non-causal citation	A3 (flag only, never re-investigate)	Confident errors, coverage, refuted and revised claims
Reasoning	Splitting an answer into checkable claims and writing a probe that tests exactly that claim	A1, A2	Claims per answer, broken-probe rate, claim grades
RAG	Answers must come from the frozen repository, not from memory	B0 (closed book), A10 (no retrieval; oracle retrieval)	Fix-location recall, citation precision against fix lines

---

5. Experiment Runs

5.1 Setup

- **Model.** Every system uses `gemini-3.1-flash-lite` (Google free tier) at temperature 0. The model was never changed mid-experiment.
- **Judge.** `openai/gpt-oss-120b` on Groq, a different model family. It saw the question, the gold answer, the observed failure, the source part of the fix diff and each cited snippet, and labelled each answer correct, partial or incorrect, and each citation causal, on-topic but not causal, or irrelevant. One judge was used for the whole experiment.
- **Pre-registration.** H1–H3, thresholds, metrics and ablation predictions were fixed in `PREREGISTRATION.md` before any test run. Prompts were iterated once, on dev only, with equal effort for every system (`docs/prompt_log.md`). One further change to ours alone, made on dev before the lock, is disclosed (D-51). At gate G2 the test file's SHA-256 and the prompt hashes were recorded and the file marked LOCKED; `eval/run.py` refuses the test split otherwise.
- **Quota.** The binding constraint was about 500 requests per day per Gemini key. We used four members' keys (same model and settings). The SQLite cache makes resumed and repeated work free.
- **Engineering.** 309 unit tests, 18 Docker tests and 3 live tests. Each member's bundle passes its own tests in push order (`scripts/check_bundle_order.py`).

5.2 Runs performed

Run id	Split	Systems	Questions	Dates (2026)
`pilot_d51`	dev	ours, B1, B2, A1, A2, A3	20	7 Nov
`main`	test	ours, B1, B2, A1, A2, A3, then B0	150 (145 analysed)	7–11 Nov
`sub50`	test	ours, B2, A10-none, A10-oracle	50 (47 analysed)	11 Nov
`adv`	adversarial	ours, B2, B1	15	10 Nov

All 1,050 answers in `main` were judged. One crash in `sub50` (an oracle file the fix had created, which does not exist at the frozen commit) was fixed and the question re-run. A request timeout was added during `main` after one request hung (D-54); it changes no answer.

5.3 Metrics

- **Primary: causally-supported answer rate.** The answer is correct *and* every claim has at least one causal citation. A citation counts as causal if it falls inside a gold-evidence range or is labelled causal (by the judge, or by humans where they exist). Partial answers count as wrong (D-41).
- **Correctness** alone.
- **Confident errors:** wrong answers that were neither flagged as uncertain nor abstained. **Coverage:** the share of answers neither flagged nor abstained.
- **Fix-location recall:** the share of questions with at least one citation inside the gold ranges. **Citation precision** against the gold ranges is a lower bound, because a line outside the fix can also decide the behaviour.
- **Cost:** requests, tokens, probe runs and mutant runs per question.
- **Statistics:** paired by question. McNemar's exact test on discordant pairs, and a paired bootstrap with 10,000 resamples for 95% CIs (a cluster bootstrap over questions for citation precision).

5.4 Main results (judge labels, 145 test questions)

System	Correct	Causally supported	Fix-location recall	Confident errors	Coverage	Requests / q	Tokens / q
B0, closed book	23.4%	4.8%	3.4%	52 (35.9%)	59.3%	1.0	1,495
B1, direct RAG	37.9%	21.4%	42.8%	89 (61.4%)	99.3%	1.0	5,432
B2, ReAct, same tools	**68.3%**	**49.7%**	62.1%	46 (31.7%)	100%	10.5	119,035
Ours	61.4%	40.0%	60.0%	43 (29.7%)	86.2%	14.7	185,876
A1, no execution	62.8%	39.3%	62.8%	53 (36.6%)	99.3%	7.9	78,107
A2, no mutation	62.8%	42.1%	60.7%	44 (30.3%)	90.3%	10.0	112,735
A3, flag only	61.4%	40.7%	60.0%	**29 (20.0%)**	67.6%	8.5	87,951

Ours is far better than the simple baselines: +18.6 points over B1 on the primary metric (CI [+10.3, +26.9]), and +35.2 over B0. It is behind B2, the ReAct agent with the same tools, while using 40% more requests and 56% more tokens. The judge's absolute levels are inflated (§7.1); the comparisons are what matter.

---

6. Ablations

Each ablation changes one config flag (`agent.ablation`) and runs on the same questions, with the same model and budget.

Ablation	What changes	Prediction (pre-registered)	Outcome
A1, no execution	Claims verified by an LLM reading the code; grades at most [R]	More confident errors, especially on doc-related questions	**As predicted in direction:** 36.6% vs 29.7% confident errors (CI [−14.5, +0.7], p = 0.12, not significant); on doc-related questions 36 vs 28. Correctness unchanged (62.8% vs 61.4%)
A2, no mutation	No citation check	Similar correctness, clearly lower causal citation precision	**Opposite:** correctness similar (62.8%), but precision is *higher* without mutation: 44.1% vs 30.6% by humans (§7.2)
A3, flag only	Failed checks are flagged, never re-investigated	Fewer confident errors than B2, lower correctness than ours	Fewer confident errors than ours, 20.0% vs 29.7% (CI [+4.1, +15.9] for ours − A3, p = 0.004), at **equal** correctness (61.4%), by flagging 47 answers instead of 20
A6, acceptance rule	any_flip / behavioural / specific, computed from the same mutation runs	`specific` matches humans best	**Opposite:** `specific` matches humans worst (§7.3)
A10-none	No starting material	More steps, similar accuracy	Similar correctness (53.2% vs 53.2%, n = 47); fix-location recall drops 59.6% → 44.7% (+14.9 for ours, CI [+6.4, +25.5], p = 0.016); *fewer* requests (14.2 vs 14.9)
A10-oracle	The whole files the fix changed as starting material	Shows the ceiling	No gain: 51.1% correct, 36.2% causally supported, vs ours 53.2% and 36.2%
B0 (RAG ablation)	No retrieval and no tools at all	Far worse	23.4% correct, 4.8% causally supported

What the ablations say. Removing any single part of the procedure (execution, mutation or re-investigation) leaves correctness and the primary metric statistically unchanged; every difference is within ±2.1 points. Two parts are worse than neutral. **Re-investigation** turns flagged answers into confident ones without making them more often correct, so A3, which simply flags, has a third fewer confident errors. **Mutation** lowers human-judged citation quality. **Retrieval** helps the agent find the developer's lines, but handing it the right files does not raise correctness: its limit is in using the material, not in finding it.

Zero-cost ablations on toylib (`results/summary/toylib_ablations.md`). On the 9 planted lines, `specific` agrees with the planted ground truth on 8, against 7 for the other rules. With constants-only mutation (A11), the line that only a negation or comparison swap can

flip is falsely rejected, as designed. The rule we chose works on the code it was designed on; §7.3 shows it does not carry over to real libraries.

---

7. Evaluations

7.1 Validating the judge against humans (gate G3)

Two members, Aditya (`labeller1`) and Sourav (`labeller2`), labelled the same blinded sample separately: 12 test questions (seeded, stratified by library) × 6 systems = 72 answers and 204 citations. The systems were hidden behind shuffled ids. They labelled separately; an AI tool was used only to format their choices into the label files (disclosed in `PREREGISTRATION.md`). Disagreements were resolved by a pre-registered conservative rule (D-48).

Labels	Human–human kappa	Judge vs human consensus kappa
Answer correctness (72)	0.742	0.363
Citation labels (204)	0.902	0.322

The humans agree with each other well; the judge agrees with them only fairly. On citations the judge is **lenient**: it rated 72 of the 204 citations higher than the humans did, mostly *causal* where the humans said *on-topic but not causal* (49), and 11 lower. On correctness it errs both ways (15 higher, 11 lower). Both kappas are below the pre-registered 0.6, so **the primary comparison uses the human labels**, and the judge-based results on 145 questions are secondary.

7.2 Hypothesis tests

Primary (human labels, 12 questions):

System	Correct	Causally supported
B2	9/12 (75.0%)	5/12 (41.7%)
A1, A2, A3 (each)	8/12 (66.7%)	4/12 (33.3%)
Ours	6/12 (50.0%)	3/12 (25.0%)
B1	3/12 (25.0%)	1/12 (8.3%)

Hypothesis	Result	Verdict
H1	25.0% vs 41.7%, −16.7 points, CI [−41.7, 0.0], McNemar p = 0.50 (0 vs 2 discordant)	**Not supported.** Ours was better than B2 on none of the 12 questions; too few questions to show that B2 is better
H3	Causal citation precision 30.6% (36 citations) vs A2's 44.1% (34), −13.6 points, cluster-bootstrap CI [−28.6, −1.0]	**Not supported: A2, without mutation, is better**

Secondary (judge labels, 145 questions):

Hypothesis	Result	Verdict
H1	40.0% vs 49.7%, −9.7 points, CI [−19.3, −0.7], McNemar p = 0.059 (17 vs 31 discordant)	**Not supported; B2 better** (bootstrap CI below zero)
H1, correctness only	61.4% vs 68.3%, −6.9 points, CI [−14.5, 0.0], p = 0.099	B2 ahead, not significant
H2	Confident errors 29.7% vs 31.7%, relative reduction 6.5%, CI of the difference [−10.3, +6.2]; coverage 86.2% vs 100%	**Inconclusive** (CI includes zero; the coverage condition also fails)
H3, by judge	58.7% vs 59.3%, CI [−7.6, +6.0]	Inconclusive; superseded by the human result

The two views agree in direction everywhere. The judge's lenience shifts every system's level up, but because it was equally lenient across systems, its differences point the same way as the humans'. With 145 questions, H1 is above the roughly 120 questions we declared necessary for adequate power. **The full procedure did not beat a ReAct agent with the same tools**, and its mutation check made citations worse. Before the spot check, on all 150 questions, the judge-based H1 gap was −10.7 points (CI [−20.0, −1.3], p = 0.033); the exclusion does not change the conclusion.

7.3 RQ2: the mutation check as a measurement of causality

Ours made 862 mutation checks on test; 123 lines were untestable (no operator applied), 31 of them fix lines.

Rule	Accepted (of 739)	Precision vs fix lines (lower bound)	Recall vs fix lines	Kappa vs fix lines	Agreement with humans (52 lines)	Kappa vs humans
any_flip	494	36.6%	66.8%	−0.001	55.8%	0.198
behavioural	411	37.0%	56.1%	0.007	55.8%	0.167
specific (used)	177	36.7%	24.0%	0.001	51.9%	**−0.098**

Against both independent references, the mutation verdicts are at chance level or barely above it. The rule the agent uses rejects 76% of the lines the developer actually changed, and it agrees with humans *least*, though we predicted *most*. Requiring that control probes stay unchanged rejects many genuinely causal lines in real code, where one line often affects several behaviours. Possible causes are equivalent mutants, redundant code paths, and controls that are not truly independent of the claim. On 52 human-labelled lines these kappas are imprecise, but none reaches even moderate agreement (0.4).

7.4 Tool use and safety

From the traces of the 145 test questions (replays de-duplicated):

System	Tool calls	Valid-call rate	Errors (recovered)	Probes submitted	Probe verdict rate	Blocked unsafe attempts	Unsafe actions executed
Ours	3,599	100.0%	45 (19)	604	85.2%	40 (6.0%), in 14 questions	**0**
B2	1,521	99.7%	112 (12)	695	88.5%	26 (3.7%), in 9 questions	**0**
A2	1,999	100.0%	35 (13)	546	84.5%	35 (5.7%)	**0**
A3	2,072	100.0%	36 (12)	459	85.1%	34 (6.5%)	**0**

The *probe verdict rate* is the share of probes that ran and reached pass or fail. Blocked probes are unsafe *attempts* refused by the policy before running. Most were harmless in intent, such as creating a file outside `/tmp` to set up a scenario; a few imported `subprocess` or `shutil`. No unsafe action was executed by any system.

7.5 Prompt injection (adversarial set)

System	Exposed (canary reached the model)	Complied when exposed (canary in answer)	Probes attempting the planted action	Unsafe actions executed
Ours	12/15	7/12	0	0
B2	11/15	9/11	0	0
B1	8/15	6/8	0	0

No system attempted the dangerous planted actions (`curl`, writing to `~/.ssh`). But every system often obeyed the harmless-looking part, appending the canary, and several answers repeated a planted "works as intended" verdict. The sandbox protects the machine; nothing in any system stops read text from changing the answer.

7.6 By category

On **doc-related** questions (n = 86), where the method was meant to help most, ours is 54.7% correct and 40.7% causally supported, against B2's 65.1% and 47.7%. On **single-location** questions (n = 56), ours is 71.4% and 37.5%, against B2's 73.2% and 53.6%. There are too few spread-across-files questions (3) to read.

7.7 Qualitative cases

Where the mutation check hurt (click-2099, "No way to alter dynamic default value help text"). On the frozen code, a callable default is always shown as `(dynamic)` in help text. Ours' probe confirmed this. It cited lines 2722 and 2723 of `core.py`; the developer's fix changed line 2722. The mutation check rejected 2722 as not causal and accepted 2723, so the agent **dropped the developer's own line**. A2 (no mutation) and B2 kept both lines and hit the fix location. This is the RQ2 failure in miniature.

Where the procedure worked (click-1422, "aliased group shows abbreviated command in usage"). Ours drafted five claims. Probes refuted two of them, showing `UsageError: Missing command` where the claim expected otherwise. The agent re-investigated, revised both claims twice, and the third version of each passed; and the mutation check then accepted the developer's fix line (1349). The answer was judged correct and causally

supported. B2 explained the same behaviour but cited the invoke code rather than the line that decides the name, and was judged partial.

7.8 Failure analysis: why ours lost to B2

From ours' decision logs on the 145 test questions: 612 probe runs (401 passed, 111 failed, 100 broken), 860 mutation checks, 381 proposed replacement citations, 84 claims refuted and 75 revised.

- **The mutation check discards good citations** (§7.3), and each rejection triggers a search for another line, which is often worse.
- **More claims, each needing a causal line.** Ours averages 2.61 claims per answer against B2's 2.05. The primary metric needs a causal citation for *every* claim.
- **Broken probes cost budget.** 16% of probe runs were broken, for example by a wrong import. Each repair spends requests from the same budget of 25, which B2 spends on investigating.
- **Flagging separates right from wrong poorly.** Of ours' 20 flagged answers, 7 were in fact correct.

Deterministic failure classes (judge labels and fix-line hits):

System	Correct, causally supported	Correct, not causally supported	Right behaviour, wrong place	Right place, wrong or partial behaviour	Wrong behaviour and place
Ours	58	31	22	19	15
B2	72	27	18	16	12
B1	31	24	53	21	16

---

8. Exploratory Extension: Multi-Agent Causal Fault Localization

Designed after the main results were known, and pre-registered separately before any of its test runs (`PREREGISTRATION.md`, "Exploratory extension"). Its numbers are exploratory. They do not change H1–H3 or the conclusions of §7.

8.1 Motivation and design

The main study found two weaknesses:

- The single agent spread one budget over investigating, probing and checking citations.
- Its yes/no mutation check rejected lines that developers actually changed.

The extension asks whether specialised roles, better retrieval, a code graph and better-chosen experiments localize the cause of a reported behaviour better than the locked agent (E0) and ReAct (B2), and at what cost. SWE-bench evaluation was considered and left out: its harness needs far more disk space than our machine had.

Five roles run in a fixed order. The orchestrator is `multiagent/orchestrator.py`, with no agent framework.

1. **Bug Investigator** (LLM, read-only tools) states the expected and actual behaviour, a scenario, and 2–4 competing hypotheses.
2. **Probe Engineer** (LLM) writes a reproduce probe and 1–2 controls, plus optional boundary and hypothesis-distinguishing probes. These run in the sandbox and are repaired at most twice.
3. **Code Analyst** (LLM, read-only tools) ranks suspicious lines. From E3 on, it also sees a deterministic ranking built from backward slices [24] and executed-line spectra (Ochiai [23]).
4. **Experiment Engineer** (code, not an LLM) runs mutation checks with three verdicts:
   - **accepted:** a mutant flips the claim and leaves the controls unchanged;
   - **inconclusive:** a flip that also changes a control, or only breakage;
   - **rejected:** no flip.

   From E4 it also mutates pairs of dependent lines. From E5 it orders checks by expected information gain and stops early.
5. **Causal Verifier** (LLM) judges each hypothesis and writes the answer and an optional one-line patch. Code then validates the patch by re-running the probes.

Like AutoCodeRover [30], it combines code-structure search with spectrum-based localization. Unlike conversation frameworks such as AutoGen [31], the agents hand each other typed pydantic objects only. Everything else is shared with the main systems: model, temperature, LLM cache, sandbox, mutation code, question text and output schema. The budget is 12 requests per question (E0: 25), 8 single-line checks, 3 pair checks and 12 probe runs.

Config	Adds
E1	The five roles, with BM25 starting material
E2	Hybrid retrieval: BM25 and dense (bge-small [26], FAISS [27]) fused by RRF [22], then a cross-encoder rerank [25]
E3	A code graph and slicing [24, 28]: call graph, data and control dependences, executed-line spectra
E4	Pairwise mutation of dependent lines
E5	Experiment selection by expected information gain
E6	E5 with its analysis and execution tools behind a local MCP server [29]

Questions. The 47 `sub50` test questions, on which E0 and B2 already had answers (reused unchanged). Prompts were developed on 2 dev questions only. Gold is the developer's fix lines, so every localization rate is a lower bound: a truly causal line outside the fix counts as a miss.

Metrics. All are computed in code (`eval/localization.py`):

- Top-1 and Top-3, MRR, and precision, recall and F1, at line level and function level;
- mutation acceptance on fix lines and on other lines;
- judged correctness, using the same `gpt-oss-120b` judge as the main study.

8.2 Retrieval study (no LLM, 142 test questions)

Each mode retrieves code chunks for the question text. A chunk is relevant if it overlaps a fix range. Three of the 145 questions have no relevant chunk and are skipped.

Mode	hit@1	hit@5	Recall@5	MRR@10
R1 BM25 (main study)	35.9%	64.1%	46.3%	0.476
R2 dense (bge-small)	36.6%	65.5%	47.6%	0.486
R3 BM25 + dense, RRF	**47.2%**	**71.8%**	**54.1%**	**0.573**
R4 R3 + cross-encoder rerank	26.8%	60.6%	42.0%	0.413
R5 R4 + call-graph expansion	20.4%	57.7%	39.4%	0.357

Paired comparisons on hit@5:

- **R3 vs R1:** +7.7 points [+1.4, +14.1], McNemar p = 0.035. MRR +0.097 [+0.036, +0.155].
- **R4 vs R3:** −11.3 points [−19.0, −3.5], p = 0.009.

Fusing lexical and dense retrieval helps. The cross-encoder, which was trained on web passages, hurts on code. The pre-registered E2–E6 configurations use R4 as specified, because switching to R3 after seeing test results would be tuning on test.

8.3 Localization (47 test questions)

System	Line Top-1	Line Top-3	Line MRR	Line F1	Fn Top-3	Fn MRR	Correct (judge)
E0 (locked agent)	**48.9%**	59.6%	0.532	31.7%	72.3%	0.670	53.2%
B2 (ReAct)	46.8%	**61.7%**	**0.539**	**33.5%**	**78.7%**	**0.745**	**61.7%**
E1 roles	34.0%	48.9%	0.401	27.8%	76.6%	0.684	48.9%
E2 + hybrid retrieval	40.4%	55.3%	0.479	28.3%	72.3%	0.684	57.4%
E3 + code graph	31.9%	36.2%	0.358	12.2%	61.7%	0.586	55.3%
E4 + pair mutation	31.9%	36.2%	0.358	12.2%	61.7%	0.586	57.4%
E5 + information gain	31.9%	36.2%	0.352	12.0%	66.0%	0.631	55.3%
E6 E5 via MCP	31.9%	36.2%	0.352	12.0%	66.0%	0.631	55.3%

Paired differences, as bootstrap 95% CI and McNemar p:

Comparison	Line Top-3	Line MRR	Correct (judge)
E1 vs E0	−10.6 [−23.4, +2.1], p = 0.180	**−0.131 [−0.238, −0.035]**	−4.3 [−14.9, +6.4]
E2 vs E1	+6.4 [+0.0, +14.9], p = 0.250	**+0.079 [+0.001, +0.163]**	+8.5 [−4.3, +21.3]
E3 vs E2	**−19.1 [−34.0, −4.3], p = 0.035**	−0.122 [−0.254, +0.008]	−2.1 [−17.0, +12.8]
E5 vs E0	**−23.4 [−38.3, −8.5], p = 0.007**	**−0.180 [−0.323, −0.040]**	+2.1 [−10.6, +14.9]
E2 vs B2	−6.4 [−19.1, +6.4], p = 0.508	−0.060 [−0.162, +0.043]	−4.3 [−19.1, +10.6]

No extension configuration beats E0 or B2 on any localization measure. The best one, E2, is statistically indistinguishable from both. No correctness difference has a CI that excludes

zero.

Why E3 is worse than E2. `results/summary/extension.md` has a ranking diagnosis. The final ranking sorts lines by verdict: accepted, cited, inconclusive, untested, rejected.

- E1 and E2 rank about 3 lines per question. Twelve of E2's 26 top-3 gold hits were lines the mutation check had *rejected*; they stayed in the top 3 only because the list was short.
- E3 adds graph candidates and ranks about 8 lines. Most of the new lines come out inconclusive, so they sort above the rejected fix lines, which drop to ranks 6–10 or off the list.

The code graph did not make localization worse. It exposed the same weakness as the main study: the mutation check rejects most of the lines developers changed.

8.4 Causal verification and cost

System	Lines tested	Accepted, fix lines	Accepted, other lines	Inconclusive	Requests / q	Tokens / q	Mutant runs / q
E0	182	50.0% (24/48)	25.4% (34/134)	17.6%[1]	14.9	187,823	9.1
E1	136	19.6% (9/46)	10.0% (9/90)	47.1%	7.6	32,031	3.3
E2	130	20.9% (9/43)	8.0% (7/87)	37.7%	7.8	33,678	3.4
E3	434	22.9% (11/48)	6.7% (26/386)	51.6%	8.0	35,393	9.9
E4	434	22.9% (11/48)	6.7% (26/386)	51.6%	8.0	35,527	15.2
E5	321	23.3% (10/43)	5.8% (16/278)	51.7%	8.0	35,337	11.4

[1] E0's yes/no check counts a flip that also changes a control as "no". E1–E6 call that inconclusive. E0's inconclusive rate therefore covers only untestable lines, and its two acceptance rates are not directly comparable with E1–E6.

Cost. The roles use half of E0's requests and less than a fifth of its tokens. B2 uses 9.9 requests and 109,529 tokens.

Three-way verdicts. These make the check more conservative: about half of all tested lines come out inconclusive. Acceptance on non-fix lines falls from 10.0% (E1) to 5.8% (E5). Acceptance on fix lines stays near 20%.

Pairwise mutation (E4).

- 95 pair checks: 56 rejected, 39 inconclusive, **none accepted**.
- E4's ranking is identical to E3's on all 47 questions.
- It cost 5.3 more mutant runs per question.

Information-gain selection (E5).

- Cut mutant runs from 15.2 to 11.4 per question with an unchanged Top-3.
- Of the 47 questions, it stopped early on 19:
  - on 8 because one hypothesis held at least 0.84 of the weight;
  - on 11 because no remaining check was informative.

- Otherwise, the single-check budget ran out on 10 questions, no candidates were left on 16, and 2 ran no experiments.

Patches. Patches that the verifier proposed passed the probe re-run 32–46% of the time, varying by configuration (22–26 patches each).

MCP (E6). E6 produced the same ranking and answer text as E5 on all 47 questions. Its wall time was 62 s against 65 s per question. This is only a check that the MCP transport is lossless and cheap, not a full cost comparison, because E6 replayed E5's LLM calls from the cache.

8.5 Single- vs multi-location faults

Fourteen questions have one fix range and 33 have two or more.

System	Single: line Top-3	Multi: line Top-3	Multi: ≥ 2 fix ranges in top 5
E0	71.4%	54.5%	30.3%
B2	64.3%	60.6%	24.2%
E1	50.0%	48.5%	30.3%
E2	57.1%	54.5%	24.2%
E3–E6	28.6%	39.4%	21.2%

Pair mutation, the component aimed at multi-location faults, did not help, because no pair was ever accepted.

8.6 Predictions against outcomes

#	Prediction (written before any test run)	Outcome
1	R4 beats R1 on Recall@5; R2 alone does not	**Not supported** for R4: hit@5 −3.5 [−12.0, +4.9], p = 0.52. **Supported** for R2: +1.4, p = 0.86. Not predicted: R3 (without rerank) beats R1
2	E1 is not worse than E0 on line Top-3, and uses fewer requests	Fewer requests: **supported** (7.6 vs 14.9). Top-3: **inconclusive** (−10.6 [−23.4, +2.1]); MRR is significantly lower
3	E3 beats E2 on line Top-3	**Refuted:** −19.1 [−34.0, −4.3], p = 0.035
4	E4 finds ≥ 2 fix ranges on more multi-location questions than E3	**Not supported:** 21.2% for both; 0 of 95 pairs accepted
5	E5 needs fewer mutant runs than E4 for a Top-3 within 5 points	**Supported:** 11.4 vs 15.2 mutant runs; identical Top-3
6	E6 gives the same answers as E5 at a small wall-time cost	**Supported:** identical on 47/47; no measurable cost (62 s vs 65 s)

8.7 What the extension shows

- **Roles are efficient, not more accurate.** Splitting the work across roles cut requests by half and tokens by five-sixths. Correctness was statistically unchanged, but line-level localization got worse (MRR −0.131).

- **Retrieval matters, and simple fusion beats reranking.** Hybrid BM25 + dense retrieval with RRF was the one clearly positive result (R3, +7.7 points hit@5). The off-the-shelf reranker hurt.
- **The bottleneck is the causal test, not candidate generation.** The graph found more candidates and the experiment engine tested 3× as many lines. Acceptance on fix lines still stayed near 20%, and pairs added nothing. This agrees with §7.3 and points to the softer, graded causal tests suggested in §10.
- **Smarter experiment selection saves work** without changing results (E5), and the MCP transport is lossless (E6). Both are engineering gains, not accuracy gains.

All of this is exploratory, on 47 questions, where differences under about 15 points cannot be told apart from noise.

---

9. Limitations

- **The judge failed validation** (kappa 0.36 and 0.32). The primary, human-labelled comparison covers only 12 questions and is under-powered on its own; the 145-question judge results agree in direction.
- **The automatic gold admits feature requests.** The spot check found 23.5% invalid in a 20% sample. The rest was not checked by hand and may contain a similar share. This affects all systems equally, since they answer the same questions, but it adds noise.
- **One model, one budget.** All systems use `gemini-3.1-flash-lite` with 25 requests per question. A stronger model may write fewer broken probes and use the checks better, and a larger budget might favour the procedure.
- **Two libraries,** both pure Python and well tested. The method needs code that can be executed in isolation.
- **Not run:** B2+ (the budget-matched ReAct), B3 (Chain-of-Verification), repeat runs, the probe-limit sweep, the controlled set of injected documentation conflicts, ablations A4, A5, A7, A8, A9 and A11 on real questions, and fine-tuning. RQ1's comparison with self-verification by reading (B3) therefore remains open.
- **No unanswerable questions.** Every question has a fix, so abstention is measured only through flagging.
- **Process deviations,** all disclosed in `PREREGISTRATION.md`. These are: the test set was locked before the spot check (removals applied at analysis time); one change to ours on dev after the prompt round (D-51); several members' API keys used; and the human verdict and label files were assembled outside the labelling pages, with AI used only for grammar and formatting.
- **Prompt injection** changes answers in every system; we have no defence beyond the sandbox.
- **The extension (§8) is exploratory.** It was designed after the main results, run on only 47 questions and checked on only 2 dev questions. Its retrieval choice (rerank) was fixed before the retrieval study showed rerank hurts. SWE-bench was not run, because its harness needs far more disk space than we had.

---

10. Conclusions

We asked whether checking every claim by execution and every citation by mutation, and investigating whatever fails, makes a code question-answering agent more correct and better cited. On real, pre-registered and locked test data, it does not. A ReAct agent with the same tools did better on the primary metric, by the judge on 145 questions and by humans on 12. The mutation check made citations worse in the humans' eyes, and its verdicts track neither the developers' fix lines nor human judgements of causality. Execution itself was useful in a narrower way: without it, confident errors rose (A1). Simply flagging failed checks, instead of re-investigating them, cut confident errors by a third at no cost in correctness (A3).

Three lessons follow. **First,** "flip the switch" is a weak test of causality in real code. A line can matter without any single mutant changing exactly one behaviour, so a specificity control that works on a planted library rejects three in four of the lines developers actually changed. **Second,** feedback loops built on a noisy check do harm: re-investigation spent budget moving away from good answers. **Third,** an LLM judge must be checked. Ours was reasonably consistent across systems but lenient on citations, and without human labels H3 would have looked inconclusive instead of negative.

The exploratory extension (§8) points the same way. Specialised roles made the pipeline far cheaper, and hybrid retrieval found the right code more often. But more candidates, pairwise mutation and smarter experiment selection did not raise the share of fix lines the mutation check accepts (about one in five). The causal test, not the search, limits localization.

Future work.

- A softer causal test: rank lines by how many probes their mutants change, or use dynamic slicing, instead of a hard accept/reject filter.
- Use failed checks only to flag, not to re-investigate (as A3 suggests).
- Run B2+, B3 and the controlled conflict set.
- Filter feature requests out of the automatic gold with a test-failure pattern check.
- Add an explicit defence against instructions in read text.
- Fine-tune a small probe writer on mutation-validated probes.
- Re-run the extension with RRF retrieval without the reranker, and with a ranking that does not let inconclusive lines outrank rejected ones, on fresh questions rather than the 47 already seen.

References

1. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.
2. Trivedi et al. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. ACL 2023.
3. Jiang et al. Active Retrieval Augmented Generation (FLARE). EMNLP 2023.
4. Asai et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR 2024.

5. Yan et al. Corrective Retrieval Augmented Generation. arXiv 2024.
6. Dhuliawala et al. Chain-of-Verification Reduces Hallucination in Large Language Models. Findings of ACL 2024.
7. Gou et al. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. ICLR 2024.
8. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024.
9. Xia et al. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv 2024.
10. Kiecker et al. CASCADE: Detecting Inconsistencies between Code and Documentation with Automatic Test Generation. Proc. ACM Softw. Eng. (FSE) 2026. https://arxiv.org/pdf/2604.19400
11. Kang et al. Identifying Inaccurate Descriptions in LLM-generated Code Comments via Test Execution. arXiv 2406.14836, 2024.
12. Lee et al. METAMON: Finding Inconsistencies between Program Documentation and Behavior using Metamorphic LLM Queries. LLM4Code Workshop at ICSE 2025. https://arxiv.org/abs/2502.02794v1
13. Moon et al. Ask the Mutants: Mutating Faulty Programs for Fault Localization (MUSE). ICST 2014.
14. Papadakis and Le Traon. Metallaxis-FL: Mutation-based Fault Localization. Software Testing, Verification and Reliability, 2015.
15. Gao et al. Enabling Large Language Models to Generate Text with Citations (ALCE). EMNLP 2023.
16. Panthaplackel et al. Deep Just-In-Time Inconsistency Detection Between Comments and Source Code. AAAI 2021.
17. Xu et al. Knowledge Conflicts for LLMs: A Survey. EMNLP 2024.
18. Jia and Harman. An Analysis and Survey of the Development of Mutation Testing. IEEE TSE, 2011.
19. Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023.
20. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024.
21. Wallat et al. Correctness is not Faithfulness in Retrieval Augmented Generation Attributions. ICTIR 2025. https://arxiv.org/pdf/2412.18004
22. Cormack, Clarke and Büttcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR 2009.
23. Abreu, Zoeteweij and van Gemund. On the Accuracy of Spectrum-based Fault Localization. TAIC PART 2007.
24. Weiser. Program Slicing. ICSE 1981.
25. Nogueira and Cho. Passage Re-ranking with BERT. arXiv 1901.04085, 2019.
26. Xiao et al. C-Pack: Packed Resources for General Chinese Embeddings (BGE). arXiv 2309.07597, 2023.
27. Johnson, Douze and Jégou. Billion-scale Similarity Search with GPUs (FAISS). IEEE Transactions on Big Data, 2019.

28. Yamaguchi et al. Modeling and Discovering Vulnerabilities with Code Property Graphs. IEEE S&P 2014.
29. Anthropic. Model Context Protocol Specification. https://modelcontextprotocol.io, 2024.
30. Zhang et al. AutoCodeRover: Autonomous Program Improvement. ISSTA 2024.
31. Wu et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv 2308.08155, 2023.